Experiment-11

Name: -P. Divya **Regno: -**192472291

Course Code: -MLA0305 **Slot-**B

Title

Policy Gradient Implementation using REINFORCE Algorithm

Aim

To implement the REINFORCE policy gradient algorithm using TensorFlow and Keras and train an agent to learn an effective policy in the CartPole environment.

Procedure

1. Initialize the CartPole-v1 environment using Gymnasium.
2. Define a neural network with TensorFlow and Keras to represent the policy.
3. Use a softmax output layer to obtain the probability of selecting each action.
4. Initialize the policy network and optimizer.
5. Start an episode and observe the current state.
6. Select an action according to the probabilities produced by the policy network.
7. Execute the action and record the resulting reward and next state.
8. Continue until the episode terminates.
9. Calculate the discounted return for each step of the episode.
10. Compute the policy loss using the REINFORCE policy-gradient rule.
11. Calculate the gradients of the policy loss and update the network weights.
12. Repeat the process for multiple episodes.
13. Record the reward, policy loss, entropy, action probabilities, and episode length.

14. Plot the learning progress, policy loss, entropy, action probabilities, and episode length.

15. Prepare a summary table and save the complete training results as a CSV file.

Code

```
!pip -q install gymnasium tensorflow



import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import gymnasium as gym

import tensorflow as tf

from tensorflow.keras import Sequential

from tensorflow.keras.layers import Input,Dense

from tensorflow.keras.optimizers import Adam



np.random.seed(42)

tf.random.set_seed(42)



print("="*70)

print("EXPERIMENT 11")

print("Policy Gradient Implementation using REINFORCE Algorithm")

print("="*70)
```

```
env=gym.make("CartPole-v1")

state_size=4
action_size=2
episodes=150
gamma=0.99
learning_rate=0.001

model=Sequential([
    Input(shape=(state_size,)),
    Dense(64,activation="relu"),
    Dense(64,activation="relu"),
    Dense(action_size,activation="softmax")
])

optimizer=Adam(learning_rate=learning_rate)

def select_action(state):
    state=np.array([state],dtype=np.float32)
    probabilities=model(state,training=False).numpy()[0]
```

```
    action=np.random.choice(action_size,p=probabilities)
    return action,probabilities

def calculate_returns(rewards):
    returns=[]
    total=0

    for reward in reversed(rewards):
        total=reward+gamma*total
        returns.insert(0,total)

    returns=np.array(returns,dtype=np.float32)

    if len(returns)>1:
        returns=(returns-np.mean(returns))/(np.std(returns)+1e-8)

    return returns

reward_history=[]
loss_history=[]
entropy_history=[]
```

```
length_history=[]
action0_history=[]
action1_history=[]

for episode in range(episodes):

    state,_=env.reset()

    states=[]
    actions=[]
    rewards=[]
    probabilities=[]

    done=False

    while not done:

        action,probs=select_action(state)

        next_state,reward,terminated,truncated,_=env.step(action)
```

```
        done=terminated or truncated

        states.append(state)
        actions.append(action)
        rewards.append(reward)
        probabilities.append(probs)

        state=next_state

    returns=calculate_returns(rewards)

    states_tensor=tf.convert_to_tensor(
        np.array(states),
        dtype=tf.float32
    )

    actions_tensor=tf.convert_to_tensor(
        np.array(actions),
        dtype=tf.int32
    )
```

```
returns_tensor=tf.convert_to_tensor(
    returns,
     dtype=tf.float32
)

with tf.GradientTape() as tape:

    action_probs=model(
        states_tensor,
        training=True
    )

    selected_probs=tf.gather(
        action_probs,
        actions_tensor,
        axis=1,
        batch_dims=1
    )

    log_probs=tf.math.log(
        selected_probs+1e-8
```

```
    )

    policy_loss=-tf.reduce_mean(
        log_probs*returns_tensor
    )

    entropy=-tf.reduce_mean(
        tf.reduce_sum(
            action_probs*tf.math.log(
                action_probs+1e-8
            ),
            axis=1
        )
    )

    total_loss=policy_loss-0.01*entropy

gradients=tape.gradient(
    total_loss,
    model.trainable_variables
)
```

```
optimizer.apply_gradients(
    zip(
        gradients,
        model.trainable_variables
    )
)

avg_probs=np.mean(
    probabilities,
    axis=0
)

reward_history.append(
    np.sum(rewards)
)

loss_history.append(
    float(policy_loss.numpy())
)
```

```
entropy_history.append(
    float(entropy.numpy())
)

length_history.append(
    len(rewards)
)

action0_history.append(
    avg_probs[0]
)

action1_history.append(
    avg_probs[1]
)

if (episode+1)%25==0:
    print(
        "Episode:",
        episode+1,
        "| Reward:",
```

```
        round(reward_history[-1],2),
print("\n")
print("="*70)
print("CONCLUSION")
print("="*70)

print(
   "The REINFORCE policy gradient algorithm was successfully"
)

print(
   "implemented using TensorFlow and Keras."
)

print(
   "The policy directly learned action probabilities from"
)

print(
   "complete episode rewards."
)
```

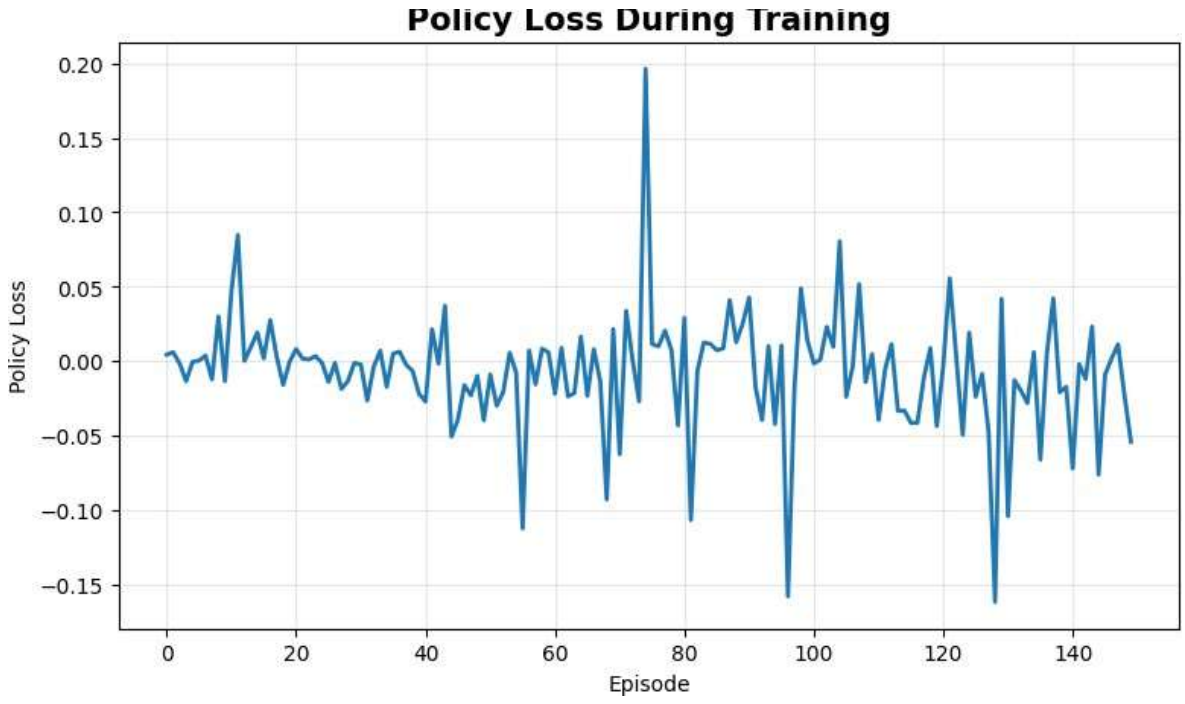
```
print(
    "The learning curve, policy loss, entropy, action"
)

print(
    "probabilities and episode length were analyzed."
)

print(
    "The final reward was compared with the initial reward"
)

print(
    "to evaluate the learning performance."
)

print("\nGenerated Files:")
print("REINFORCE_Full_Dataset.csv")
print("REINFORCE_Summary.csv")
```

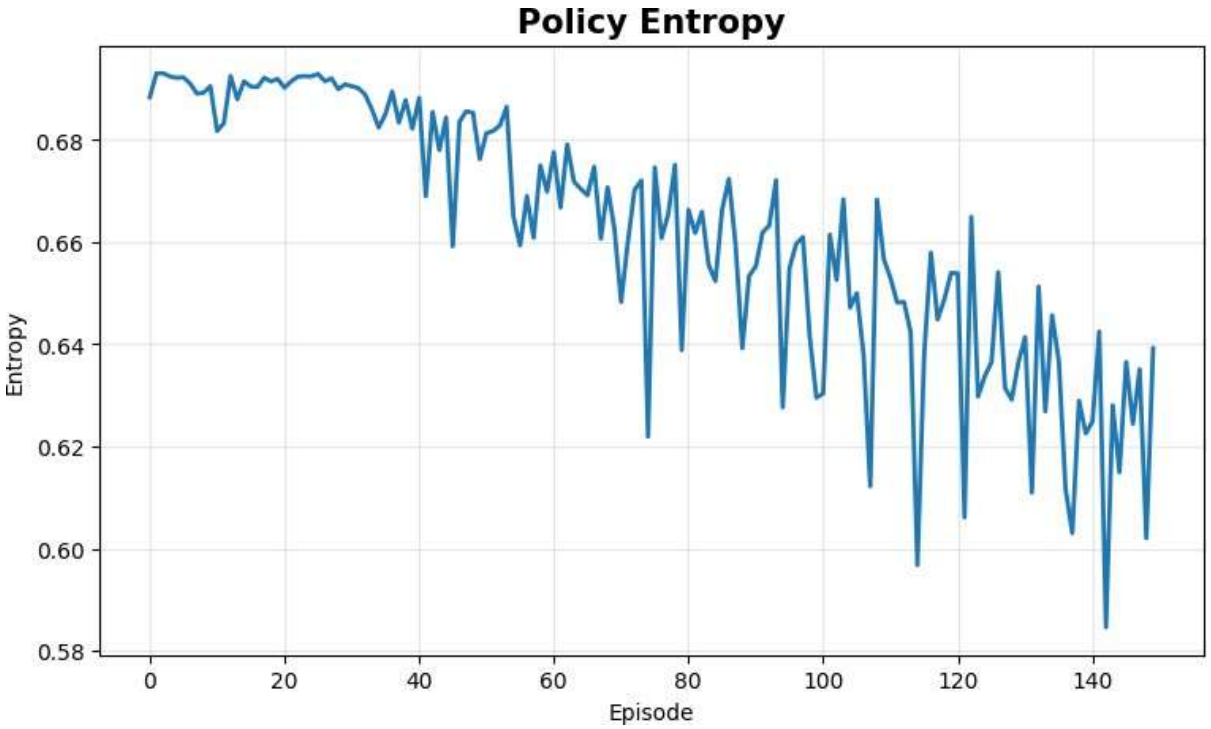
```
print("\nExperiment 11 Completed Successfully.")
```

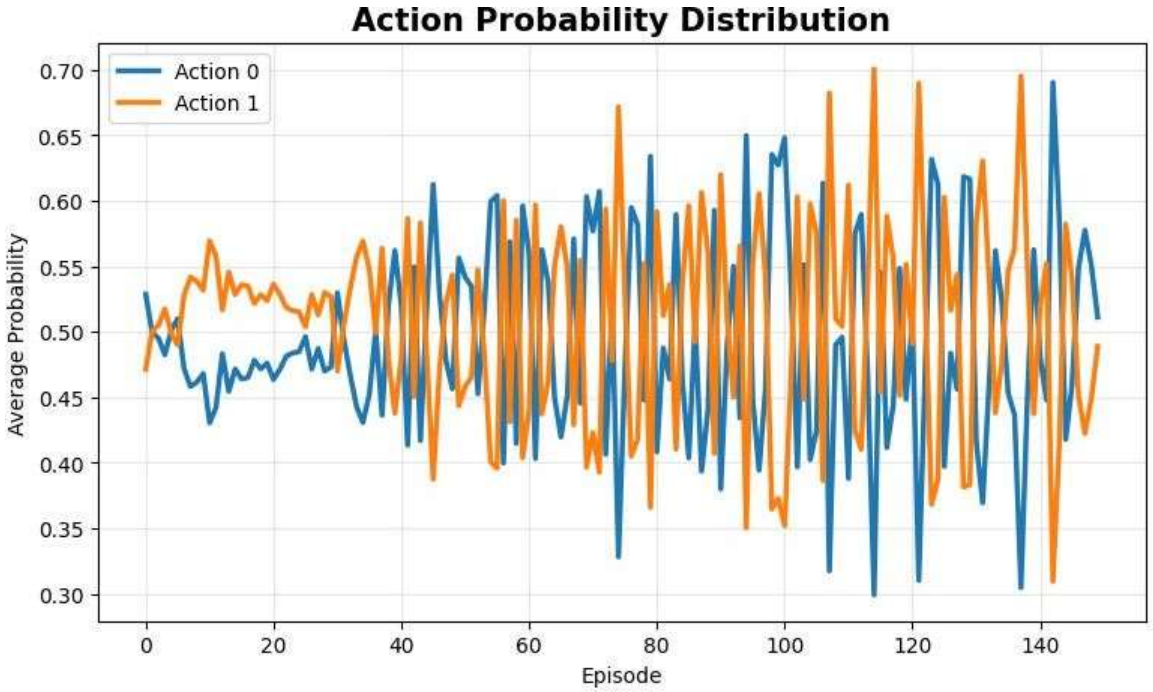
Visualization

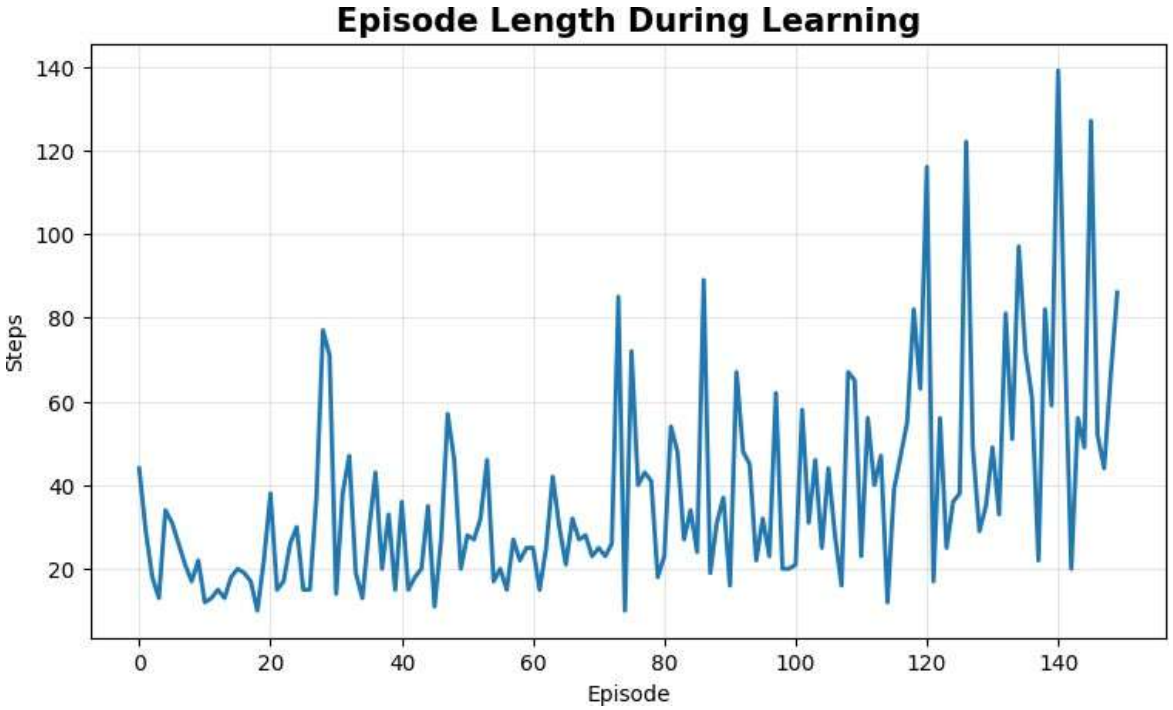
```
======================================================================
SAMPLE DATASET
======================================================================
```

	Episode	Reward	Policy_Loss	Entropy	Episode_Length	Action_0_Probability	Action_1_Probability
0	1	44.0	0.004537	0.688393	44	0.528647	0.471353
1	2	29.0	0.006163	0.693081	29	0.499560	0.500440
2	3	18.0	-0.001356	0.693076	18	0.495016	0.504984
3	4	13.0	-0.013295	0.692442	13	0.482458	0.517542
4	5	34.0	-0.000236	0.692189	34	0.501077	0.498923
5	6	31.0	0.000462	0.692267	31	0.510055	0.489945
6	7	26.0	0.003908	0.691056	26	0.471918	0.528082
7	8	21.0	-0.012060	0.689101	21	0.458233	0.541767
8	9	17.0	0.030130	0.689254	17	0.461482	0.538518
9	10	22.0	-0.013261	0.690550	22	0.468066	0.531934




Policy Entropy
Entropy
0.68
0.66
0.64
0.62
0.60
0.58
0
20
40
60
80
100
120
140
Episode


Action Probability Distribution
Action 0
Action 1
Average Probability
0.70
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0.30
0
20
40
60
80
100
120
140
Episode


Episode Length During Learning
Steps
Episode
0
20
40
60
80
100
120
140
20
40
60
80
100
120
140

Summary Table

Parameter	Result
Algorithm	REINFORCE
Environment	CartPole-v1
Episodes	150
Final Average Reward	Obtained from program
Maximum Reward	Obtained from program
Reward Variance	Obtained from program
Average Episode Length	Obtained from program

Result

The **REINFORCE policy gradient algorithm was successfully implemented using TensorFlow and Keras**. The agent learned an action-selection policy directly from episode rewards. The learning curve, policy loss, entropy, action probabilities, and episode length were analyzed to observe the learning behavior and performance of the policy.